UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 SPECIAL TOPIC IN DATA ENGINEERING

SECTION 01

AGENTIC AI TUTORIAL

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 5

Student Name	Matric No.
GOE JIE YING	A23CS0224
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187

1. AI-Assisted Data Engineering

Data engineers spend a significant amount of time coding, debugging, and maintaining ETL pipelines. AI-assisted data engineering helps automate many of these repetitive tasks, including pipeline creation, monitoring and optimisation, thereby improving productivity and reducing development time.

2. Choice of AI Agent: Express (Nexla)

In this tutorial, Express (Nexla) was selected to build the data pipeline instead of Bauplan, Claude Code, Airia, and HighByte Intelligence Hub. Several factors were evaluated to determine the most suitable solution for the use case, including alignment with the objective of building a simple data pipeline from scratch using Artificial Intelligence (AI), cost considerations, and implementation complexity. A summary of the evaluation is shown in Table 1.

Table 1: Evaluation comparison of the selected technologies

	Suitability	**Costs**	**Complexity**
Bauplan	Not suitable	N/A	N/A
Claude Code	Suitable	Requires paid subscription	Moderate
Airia	Somewhat suitable	Free plan available	High
HighByte	Somewhat suitable	Free trial available	High
Express	Suitable	Free plan available	Low

Bauplan is a data platform that provides version control capabilities such as branching, validation, atomic merges, rollback and isolated execution. It is designed to support agent-safe data operation. Therefore, the implementation of Bauplan is coupled with other development tools and AI agents such as Claude Code. As a result, it was considered overly complex for the objective of creating a simple data pipeline and was not selected.

Claude Code is an agentic software engineering environment that can inspect codebases, run commands, edit files, invoke Model Context Protocol (MCP) tools and automate development workflows. Claude Code is the best choice in this tutorial, plus, it is similar to the data pipeline that was implemented in Python code as well. However, Claude Code requires a paid subscription, therefore, it was not selected.

Airia is an enterprise AI platform used to build, deploy, and manage AI agents across organisations. Although it offers a free plan, its implementation involves additional configuration and governance features that are more suitable for enterprise-scale deployments. Consequently, it was considered more complex than required.

HighByte Intelligence Hub is a DataOps platform designed to collect, model, transform and contextualise industrial data from multiple sources. While it provides powerful capabilities for industrial and manufacturing environments, it is not specifically targeted at simple AI-assisted pipeline creation. In addition, its setup process is relatively complex and may require additional software installation. Therefore, it was not selected.

Express by Nexla is a platform that enables users to build data pipelines, connectors, and MCP integrations using natural language prompts. This closely aligns with the objective of creating a simple data pipeline using AI assistance. Furthermore, it offers a free plan and provides a user-friendly interface with minimal setup requirements. For these reasons, Express was selected as the technology for this tutorial.

3. Use Case Diagram

The project includes several actors and key use cases. Figure 3.1 presents the use case diagram of the AI-Assisted E-Commerce Analytics Pipeline.

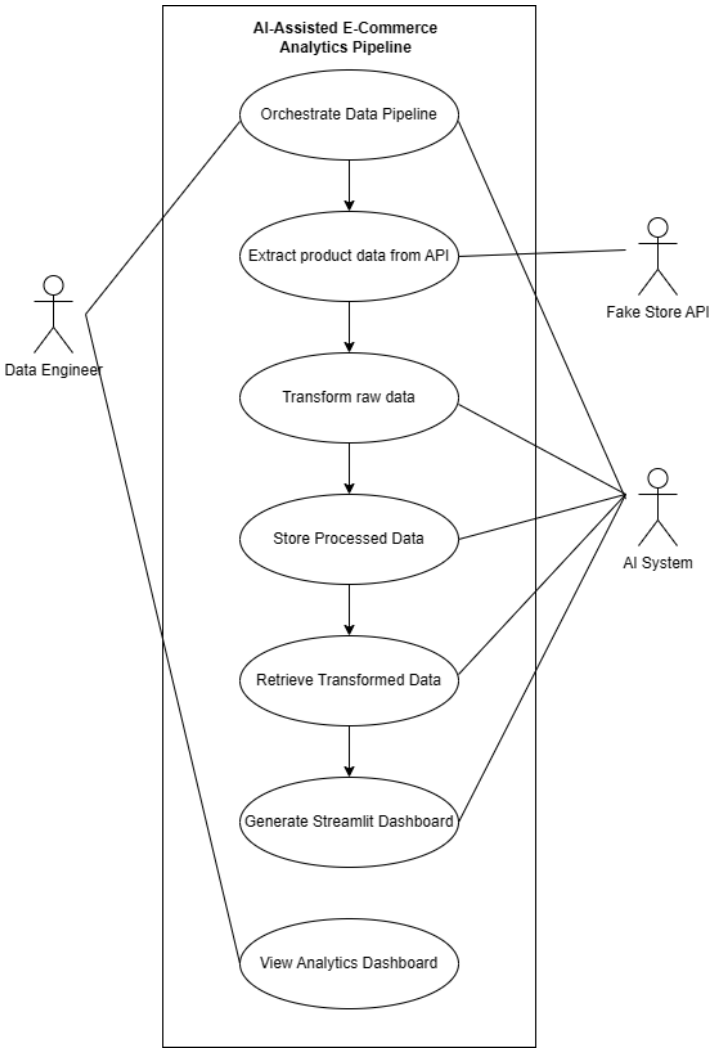


Figure 3.1 Use Case Diagram for AI-assisted E-commerce Analytics Pipeline

4. Steps: Prompt and Code Working with Data

4.1 Overview of Prompt Engineering Approach

This project was developed using an iterative agentic AI approach, where the data engineering pipeline was progressively built through structured prompts. The development process began with a traditional data warehouse design using Snowflake, but due to credential and access limitations, the architecture was later adapted to a Streamlit-based analytics system.

This change shows flexibility in pipeline design and reflects real-world data engineering constraints, where systems are often adjusted based on infrastructure availability.

The final pipeline follows this structure:

API → Python ETL → Transformed Dataset → Streamlit Dashboard

4.2 Initial Design Prompt (Snowflake-Based Architecture)

Prompt Used:

```
None
You are an expert data engineer AI agent.
Your task is to build an automated data pipeline for e-commerce analytics.

🎯 Goal:
Build a pipeline that extracts product data from an API, transforms it into
business insights, and loads it into Snowflake for analysis.

📦 Data Source:
Use Fake Store API: https://fakestoreapi.com/products

🏁 Destination:
Snowflake table: product_sales_analytics

⚙️ Pipeline Steps:
1. Data Extraction
Fetch all product data from API
Extract: id, title, price, category, rating.rate
2. Data Transformation
Create new features:
estimated_sales_qty = random integer between 10–200
estimated_revenue = price × estimated_sales_qty
performance_label:
rating ≥ 4.5 → “High Performer”
rating 3.0–4.49 → “Medium Performer”
rating < 3.0 → “Low Performer”
clean category names (lowercase, trimmed)
3. Data Loading
Load processed data into Snowflake table: product_sales_analytics
```

```
Ensure:
idempotent loading (no duplicate product_id inserts)
primary key = product_id

🧱 Output Required:
Pipeline architecture explanation
Step-by-step workflow
Generated transformation code (Python or SQL)
Snowflake schema (CREATE TABLE statement)
Sample output table (5 rows)
```

Outcome:
The AI extracted data from a fake store API and generated the foundational ETL data engineering pipeline with Snowflake database schema (product_sales_analytics). The business logic for revenue estimation and product classification were done in transformation step and load into snowflake schema.

Resulting Architecture:
At this stage, the system was designed as:

API → Python ETL → Snowflake → Analytics Layer

4.3 Issue Encountered (Credential Limitation)
During implementation, Snowflake integration could not be completed due to missing credentials and authentication setup issues as listed below:

- Snowflake connection could not be established
- Data loading step was not executable
- Pipeline execution was blocked at the storage layer

Decision:
To resolve the credential limitation, the system was redesigned using a lightweight local data processing approach. Instead of storing processed data in Snowflake, all transformations were handled in-memory using pandas DataFrames. Finally, the dataset was directly consumed by a Streamlit dashboard for real-time visualization.

4.4 Architecture Modification Prompt (Switch to Streamlit)
Prompt Used:

```
None
skip snowflake setup instead i want a streamlit dashboard
```

Outcome:
Snowflake was removed completely from the pipeline and the processed data will be accessed via API from Streamlit as the visualization layer for the system. As a result, the

focus of the data pipeline shifted from a data warehousing approach to a lightweight real-time analytics architecture.

Implementation Code:

The AI generated a complete Streamlit code application that supports both live and sample data sources. The code is presented in the below code block.

```python
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import requests
from datetime import datetime

# Page configuration
st.set_page_config(
    page_title="E-Commerce Product Analytics",
    page_icon="📊",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Nexla API Configuration
NEXLA_API_URL = "https://dataops-sync.nexla.io/sync-e298e8/428649"
NEXLA_API_KEY = "YOUR_API_KEY_HERE"  # Replace with actual API key

def fetch_data_from_nexla():
                        "SIMPLIFIED FOR THE FUNCTION"

# Sample data fallback (for demo purposes)
def get_sample_data():
    """Generate sample data matching the transformed schema"""
    return pd.DataFrame([
          {"id": 1, "title": "Fjallraven - Foldsack No. 1 Backpack", "price": 109.95,      "category":      "men's      clothing",      "rating_rate":      3.9, "estimated_sales_qty":  80,  "estimated_revenue":  8796.00,  "performance_label": "Medium Performer"},
          {"id": 2, "title": "Mens Casual Premium Slim Fit T-Shirts", "price": 22.30, "category": "men's clothing", "rating_rate": 4.1, "estimated_sales_qty": 81, "estimated_revenue": 1806.30, "performance_label": "Medium Performer"},
                            "SIMPLIFIED FOR THE REST"
# Main Dashboard
def main():
                        "SIMPLIFIED FOR THE MAIN FUNCTION"

file_name=f"ecommerce_analytics_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv",
        mime="text/csv"
```

```
    )

if __name__ == "__main__": main()
```

Code Explanation:

The generated Streamlit application implements a full end-to-end analytics dashboard with the following key components:

- Data Ingestion Layer: A function (fetch_data_from_nexla) is used to retrieve real-time data from the Nexla Sync API using REST requests.
- Fallback Mechanism: A sample dataset (get_sample_data) is included to ensure the dashboard remains functional even when API access fails or credentials are missing.
- Data Processing Layer: Data is converted into pandas DataFrames for filtering, aggregation, and transformation.
- Visualization Layer: Streamlit combined with Plotly is used to generate interactive charts, KPIs and dashboards.

This design allows the system to operate in both development mode (sample data) and production mode (live API data).

Resulting Architecture:

The AI restructured the system into:

API → Python ETL → Pandas DataFrame → Streamlit Dashboard

4.5 Identified Issue in Streamlit Implementation

During implementation of the Streamlit dashboard, an issue was identified in the generated code. Although the system was designed to support live data integration from the Nexla Sync API, the dashboard does not automatically use the real API data by default.

This issue is summarized as follows:

- The dashboard defaults to sample data instead of live API data
- Live API data requires manual selection from the sidebar
- Real-time pipeline execution is not automatically triggered

Decision:

To address this limitation, the system design was reviewed and identified that the sample dataset was included as a fallback mechanism for development purposes. However, this fallback became the default execution path.

To improve the system:

- The live API connection should be prioritised as the primary data pipeline
- Sample data should be restricted to fallback usage only

4.6 Architecture Refinement Prompt (Enforce Fake API Data Usage)
Prompt Used:

```
None
I want to use data in fake API, do not use the mock dataset for dashboard in
the app.py.
```

Outcome:
The AI revised the Streamlit implementation to prioritise real-time data retrieval from the Fake Store API via the Nexla Sync API connection and primarily depends on live API data for analytics. The updated system reduces reliance on the sample dataset and is only retained as a fallback mechanism in cases where the API request fails or returns no data.

As a result, the data flow was further aligned with a real-time analytics architecture, where the primary source of truth is the external API rather than static mock data.

Implementation Code:

```python
def fetch_from_nexla_pipeline():
    response = requests.get(NEXLA_SYNC_API_URL, timeout=30)

    if response.status_code == 200:
        data = response.json()

        # Extract transformed pipeline output (Nexla rawMessage)
        records = []
        for item in data:
            if 'raw_message' in item:
                records.append(item['raw_message']['output']['rawMessage'])

        df = pd.DataFrame(records)
        return df
```

Code Explanation:
Nexla pipeline output is now the default and primary data source. The mock data is fully removed from the execution path and serves as the fallback only.

Resulting Architecture:

Fake Store API → Nexla Transformation Pipeline → Sync API → Streamlit Dashboard

As a result, the system now achieves a complete end-to-end real-time analytics architecture with no dependency on local mock datasets.

4.7 Final Pipeline Output

The final implemented pipeline produces a real-time e-commerce analytics dashboard powered by live data from the Fake Store API. Product data is extracted via API, transformed using Python and visualised through a Streamlit dashboard.

The final architecture operates as a lightweight real-time analytics pipeline:

Fake Store API → Python Transformation → Pandas DataFrame → Streamlit Dashboard

4.8 Final Prompt Engineering Outcome

As part of the iterative development process, an additional prompt was used to request the AI to generate a complete technical specification of the system. The initial prompt produced a valid pipeline design, but it was later refined to better align with a lightweight Streamlit-based architecture.

Through refinement on initial prompt, unnecessary components, such as Snowflake integration, was removed, resulting in a cleaner and more practical implementation. The refinement is shown as below:

```
None
You are an expert data engineer AI agent.
Your task is to build an automated data pipeline for e-commerce analytics.

## Goal:
Build a pipeline that extracts product data from an API, transforms it into
business insights, and displays into a streamlit dashboard.

## Data Source:
Use Fake Store API: https://fakestoreapi.com/products

## Destination:
Streamlit dashboard

## Pipeline Steps:
1. Data Extraction (do it in express)
Fetch all product data from API
Extract:
id
title
price
category
rating.rate

2. Data Transformation (do it in express)
Create new features:
estimated_sales_qty = random integer between 10–200
```

```
estimated_revenue = price × estimated_sales_qty
performance_label:
rating ≥ 4.5 → “High Performer”
rating 3.0–4.49 → “Medium Performer”
rating < 3.0 → “Low Performer”
clean category names (lowercase, trimmed)
3. Data Analytics

Load processed data for analysis in streamlit dashboard:

Ensure:
idempotent loading (no duplicate product_id inserts)
primary key = product_id

## Output Required:
Pipeline architecture explanation
Step-by-step workflow
Generated transformation code (Python or SQL)
stream lit dashboard
```

The final generated system successfully demonstrated the same data pipeline use case within a new architecture using live API data, Python-based transformation and Streamlit visualisation.

This shows that consistent prompt iteration can effectively reconstruct and optimize a data engineering pipeline for the same analytical objective in a simplified deployment environment.

5. Integration of AI in Pipeline

This section explains the process of creating the e-commerce analytic pipeline by express.dev ai from the extraction of data from API to the complete dashboard building. Figure 5.1 shows the pipeline architecture structure that is implemented by express.dev ai.

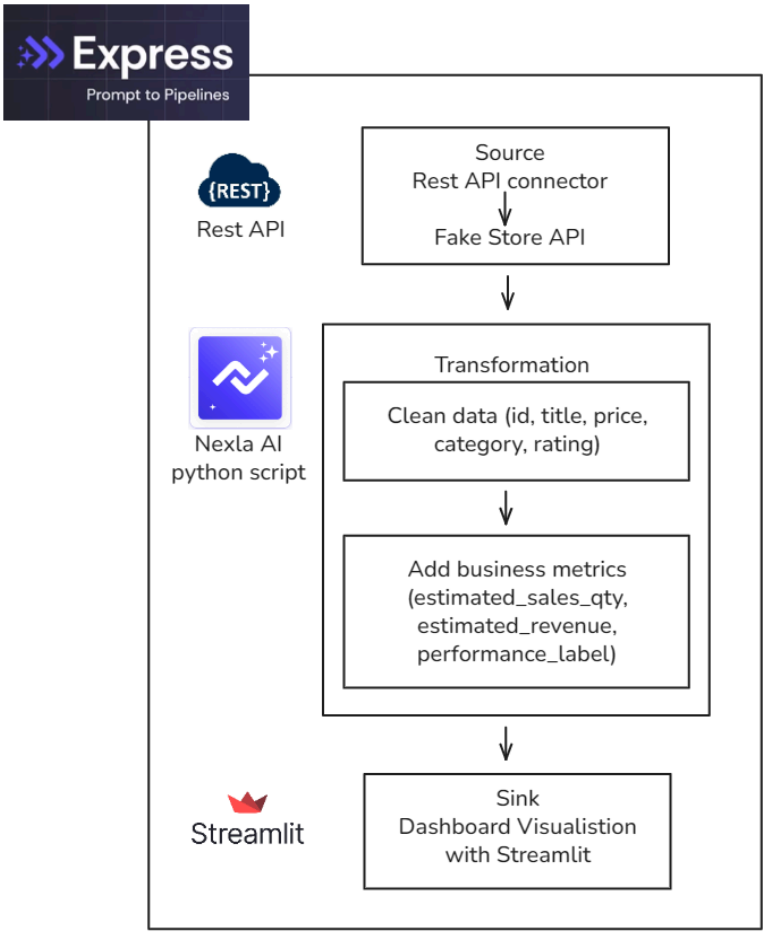


Figure 5.1 Pipeline Architecture Structure of e-commerce analysis

5.1 Setup Rest API Source

Rest API connector was automatically set up by Express.dev AI to retrieve product data from the Fake Store API. Since the API did not require authentication, a credential with auth.type: NONE was created and validated by AI. The source was then created and activated, then data was successfully extracted and verified. The flow canvas was then created to display the pipeline flow created in the next steps.

5.2 Data Transformation

The imported source was inspected and the available fields such as product ID, title, price, category and product rating were identified. A transformation plan was created based on the required analytics objective, which was to flatten the nested JSON structure from the raw source data and extract only the relevant fields. Several process was carried out to transform the data:

- Rename id to product_id
- Extract title and price as-is

- Transform category data to lowercase and remove spaces
- Flatten nested rating.rate to top-level field
- Renamed the flatten rating.rate to rate
- Check and remove null data

The first transformation is automatically executed and verified by AI. A second transformation is created to add business metrics into the record which consists of estimated_sales_qty, estimated_revenue, performance_label. The business metrics were added with the following logic:

- estimated_sales_qty: generate random integer between 10-200
- estimated revenue: calculate price × estimated_sales_qty, rounded in two decimal point
- performance_label: High performer if rate >= 4.5, Medium Performer if rate >= 3.0 and rate < 4.5, Low Performer if rate < 3.0

The final output of the transformation consists of 8 fields: product_id, title, price, category, rate, estimated_sales_qty, estimated_revenue and performance_label.

5.3 Setup Streamlit Destination

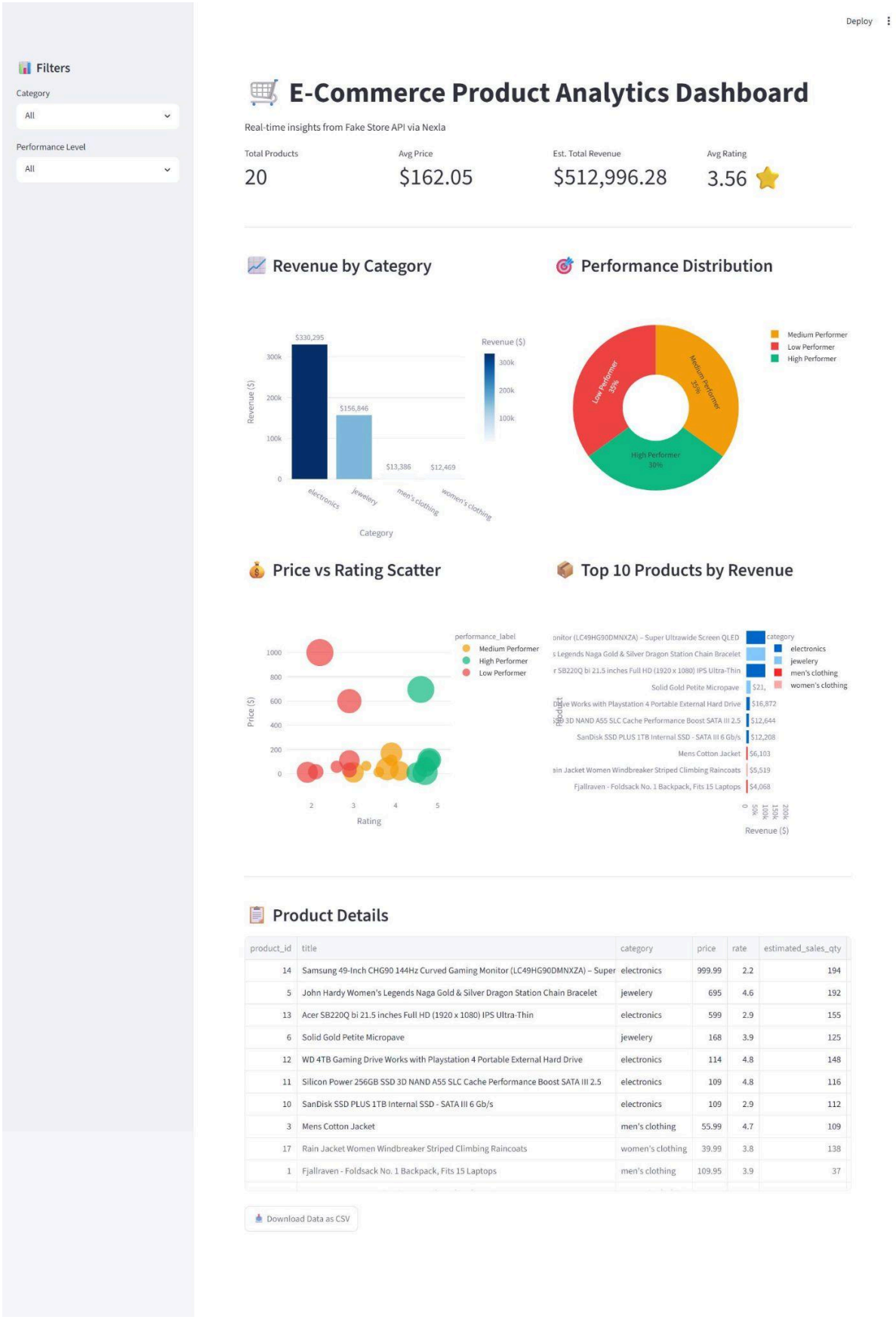
A python code app.py is generated for data retrieval, filtering, KPI calculation, chart creation and data visualization. Nexla Sync API is used as a read-only rest endpoint to return the latest data. The pipeline is automatically scheduled to run daily at 9 a.m.. The created dashboard will display:

- Real-time metrics (total products, avg price, revenue, ratings)
- Revenue by category bar chart
- Performance distribution pie chart
- Price vs rating scatter plot
- Top 10 products by revenue
- Interactive data table with download option

6. Results & Explanation

The dashboard was successfully created by AI and all expected output can be viewed in the local hosted streamlit web interface, as shown in figure 6 below. The dashboard successfully retrieved the transformed data generated from the pipeline and presented it through interactive visualizations and summary metrics.

The dashboard displays all visualizations and metrics successfully which demonstrates that the data extraction, transformation and loading processes were functioning correctly. The transformed data was accurately transferred from the pipeline to the dashboard without data loss or schema inconsistencies. Therefore, the results confirm that the AI-developed pipeline successfully converted raw API data into meaningful business insights through an automated analytics workflow.



product_id	title	category	price	rate	estimated_sales_qty
14	Samsung 49-Inch CHG90 144Hz Curved Gaming Monitor (LC49HG90DMNXZA) – Super	electronics	999.99	2.2	194
5	John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet	jewelery	695	4.6	192
13	Acer SB220Q bi 21.5 inches Full HD (1920 x 1080) IPS Ultra-Thin	electronics	599	2.9	155
6	Solid Gold Petite Micropave	jewelery	168	3.9	125
12	WD 4TB Gaming Drive Works with Playstation 4 Portable External Hard Drive	electronics	114	4.8	148
11	Silicon Power 256GB SSD 3D NAND A55 SLC Cache Performance Boost SATA III 2.5	electronics	109	4.8	116
10	SanDisk SSD PLUS 1TB Internal SSD - SATA III 6 Gb/s	electronics	109	2.9	112
3	Mens Cotton Jacket	men's clothing	55.99	4.7	109
17	Rain Jacket Women Windbreaker Striped Climbing Raincoats	women's clothing	39.99	3.8	138
1	Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops	men's clothing	109.95	3.9	37

Figure 6 Final E-commerce Product Analytic Dashboard

7. Reflection

Lam Yoke Yu

In this project, an AI platform, Express, was used to build a data pipeline. Data from the Fake Store API were loaded into Express, transformed, and then loaded into a Streamlit dashboard via an API.

Throughout this tutorial, several data platforms that support AI-assisted development, agentic AI, or data pipeline creation were explored. Although only one platform was implemented in this tutorial, it provided exposure to the AI solutions available for data pipeline development and the capabilities offered by these platforms. It also enabled us to explore the potential of AI and how it can assist in software and data development.

Despite the simplicity of the pipeline developed in this tutorial, several challenges were encountered during the implementation process. One of the main challenges was the inability to connect successfully to other services. Express provides integration options with various cloud services, including cloud storage platforms such as Google Drive and Dropbox, as well as data platforms such as Snowflake. However, errors occurred when configuring credentials, and some services required private keys could not be shared. Express also supports destinations such as email, but emails were not successfully sent after the pipeline was executed. There may have been several factors contributing to these connection failures. As a result, a Streamlit dashboard was used to present the output instead. Consequently, most of the transformations and calculations were shifted to the Streamlit application code.

Future improvements could include implementing a wider variety of transformations and extending the pipeline to handle more complex datasets and workflows.

Overall, this project provided exposure to the use of AI, particularly agentic AI in data pipeline development. It broadened our perspective on the potential of AI and opened up new possibilities for its application beyond the commonly used generative AI.

Goe Jie Ying

Through this tutorial, a deeper understanding was gained on how agentic AI can be applied in real-world data engineering pipelines. Initially, generative AI was used mainly as a supplementary tool to find solutions when facing issues during the development of data engineering pipelines using specific tools such as Azure. This approach was useful but it was often time-consuming when there was limited familiarity with the tools, as additional effort was required to explore configurations and environment setup.

After completing this project, it became clear that there is a faster and more efficient approach to building a complete pipeline through structured AI prompting. However, several challenges were also identified, particularly the issue of AI hallucination, where generated outputs may not always be fully accurate or directly applicable to real systems.

Instead of fully relying on AI-generated solutions, domain knowledge and critical thinking remain essential. All AI-generated outputs should be validated before implementation in real projects, and sensitive information such as credentials should never be shared with AI systems.

In addition, future improvements should focus on designing more precise and constrained prompts to reduce irrelevant or incorrect outputs. Greater emphasis should also be placed on validating pipeline logic against real system behavior rather than relying solely on generated code. Strengthening technical knowledge in data engineering tools will further enhance the ability to evaluate AI outputs and build more robust, reliable and production-ready pipelines.

Tan Yi Ya

This project provided valuable exposure to the application of agentic AI in data pipeline development through the use of Express. The platform enabled the construction of a complete pipeline involving data ingestion, transformation and delivery, while demonstrating how AI can assist in automating and accelerating various stages of the development process. Through this experience, a broader understanding was gained regarding emerging AI-driven technologies and their potential applications in data engineering workflows.

During the implementation process, several technical challenges were encountered, particularly in configuring pipeline components and ensuring successful integration between services. Addressing these issues required extensive debugging, including the analysis of system logs, validation of configurations and troubleshooting of data flow processes. These activities provided practical experience in identifying and resolving AI generated pipeline-related problems while improving understanding of the underlying architecture.

The project also highlighted the role of agentic AI as more than a code-generation tool. Its ability to assist with pipeline creation, workflow automation and problem-solving demonstrated its potential to enhance developer productivity. However, the experience emphasized that human oversight remains essential to validate AI-generated outputs and ensure the reliability and accuracy of the final solution. Overall, the project strengthened both technical knowledge and awareness of the growing significance of AI-assisted development in modern data engineering practices.